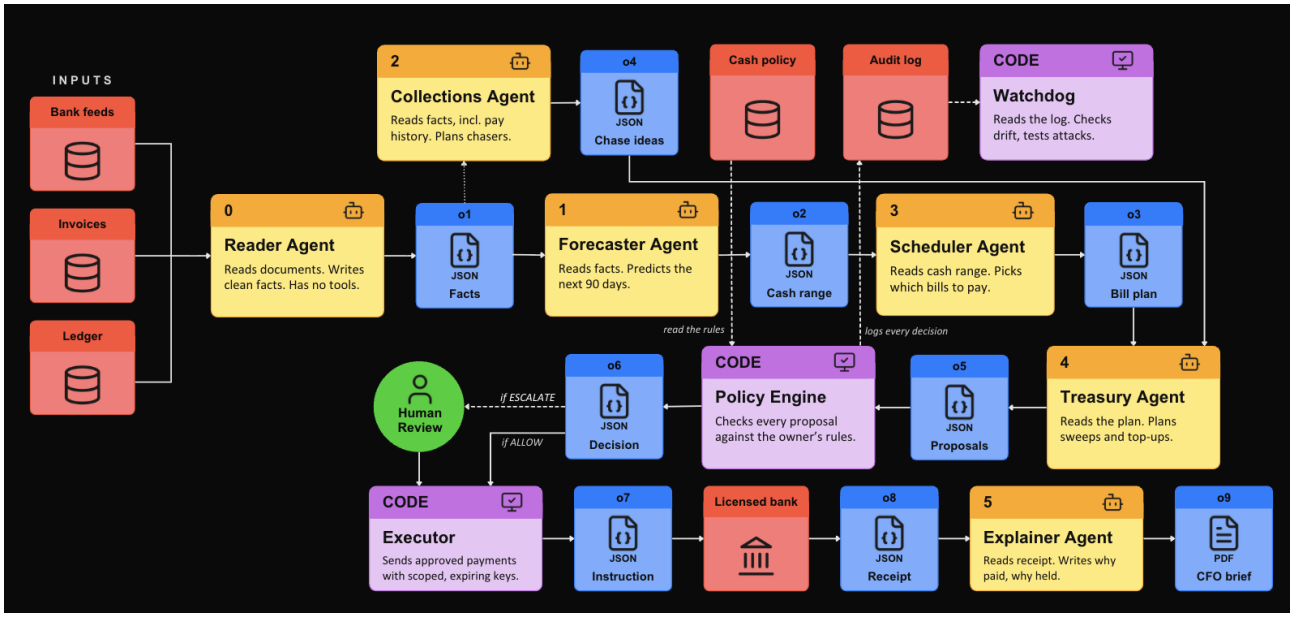
Helm technical documentation (as built)

Helm is a non custodial decision layer for small businesses. It pays a bill only when the forecast says the business stays above its minimum cash balance, and it keeps a tamper evident record of why. Every statement below matches the code, and each names the file it comes from. All business data is synthetic and the default bank is a simulation called the Demo bank.



1 System architecture

1.1 Layers

Layer	What it is	Where
Web	Next.js app with seven screens, an approval card, a simulator drawer, English and Bahasa Malaysia copy in one file	`apps/web/src`, copy in `apps/web/src/i18n/dict.ts`
API	FastAPI service. One router for screens, one for the bank, typed responses that generate the web client	`services/api/helm/main.py`, `helm/api/routes.py`, `helm/api/schemas.py`
Pipeline	A plain Python orchestration that runs the agents in a fixed order. A LangGraph graph declares the same topology for the agent view	`helm/pipeline.py`, `helm/graph.py`
Agents	Modules with one interface, `run(input) -> typed output`, and a deterministic fixture mode	`helm/agents/`
Code that holds authority	Policy Engine, Executor, Watchdog, mock bank	`helm/engine/policy.py`, `helm/executor.py`, `helm/watchdog.py`, `helm/bank/`
Data	One database with separate tables. SQLite for a one file demo, Postgres 16 for compose	`helm/models.py`, `helm/db.py`
Chain	An event only Solidity contract and a Python client	`contracts/AuditAnchor.sol`, `helm/anchor.py`

1.2 Agents and hand offs

Agents hand off through typed database records, in a fixed order. Only code can authorise money.

Step	Component	Reads	Writes	File
0	Reader Agent, no tools	documents	facts (`documents`, `invoices`, `bills`)	`agents/reader.py`, `ingest.py`

Step	Component	Reads	Writes	File
1	Forecaster Agent	facts	`cash_range`	`agents/forecaster.py`, `forecast/montecarlo.py`
2	Collections Agent	facts and pay history	`chase_ideas`	`agents/collections.py`
3	Scheduler Agent	`cash_range`	`bill_plan`, proposals	`agents/scheduler.py`, `optimiser/scheduler.py`
4	Treasury Agent	`bill_plan`	proposals for sweeps and top ups	`agents/treasury.py`
	Policy Engine, code	proposals and rules	`decisions` with a trace	`engine/policy.py`
	Human review	decisions that are ESCALATE	the owner's choice	`api/routes.py` (`owner_choice`)
	Executor, code	decisions that are ALLOW	`instructions`, `receipts`	`executor.py`, `bank/service.py`
5	Explainer Agent	receipts and the audit log	explanations and the CFO brief	`agents/explainer.py`, `brief.py`
	Watchdog, code	the audit log	`alerts`, kill switch	`watchdog.py`

There are two run modes. A full run does everything. A gate run only re-forecasts and re-decides, and it is used on the day a payment would go, so a payment is always rechecked against current cash (`pipeline.run_pipeline`, `simulator._release_due`).

1.3 Data stores

Table	Purpose
`customers`, `suppliers`, `invoices`, `bills`, `balances`, `documents`	Facts, each with a source and a confidence
`rules`	The owner's cash policy and mandate. Agents get a frozen copy, never a session that can write it
`cash_range`, `chase_ideas`, `bill_plan`	Agent outputs that the next step reads
`proposals`, `decisions`, `instructions`, `receipts`	The payment chain
`audit_log`	Append only, hash chained
`anchors`	Hashes written to a public chain, with transaction hashes
`sim_truth`, `sim_future`, `sim_state`	The hidden simulated world. No agent reads these

On Postgres, `install_pg_roles` in `helm/db.py` creates a role `helm_agent` that can read the rules but cannot write them and can only append to the audit log. This is tested by `test_agent_role_cannot_write_rules_or_edit_the_audit_log`.

2 Core algorithms

2.1 Monte Carlo cash forecast (`forecast/montecarlo.py`)

The forecaster simulates 1,000 futures of the next 90 days. The random generator is seeded from the seed 2026 and the date, so a run is repeatable.

Open invoices have a known amount and an uncertain payment day. For each customer the forecaster learns from the delays of past paid invoices, that is the paid date minus the due date, over the last 365 days. With n observations for the customer it draws from the customer's own delays with probability `n / (n + 2)` and from all customers otherwise, then adds a small amount of noise. A customer message that a payment will be late shifts the draw and widens it. An invoice that is past its sampled day and still unpaid is paid after one plus an exponential draw with a mean of eight days.

Invoices that have not been issued yet are projected from each customer's usual gap between invoices and the log of their amounts. Supplier bills not yet received are projected the same way. Payroll and statutory payments are known

dates and are not random. The result is a matrix of cumulative cash flow per future and per day, and bands are taken from its percentiles (`ScenarioSet.bands`).

2.2 The chance constraint

A payment is released only if

```
P( min over t <= H of balance_t >= B , given we pay ) >= 1 - alpha
```

where `B` is the owner's minimum balance, `H` is 90 days and `alpha` is the owner's risk tolerance. The code is `ScenarioSet.p_ok`, which takes the lowest balance of each simulated future and returns the share of futures where it stays at or above `B`.

2.3 Earliest safe date (`ScenarioSet.earliest_safe_idx`)

Paying later is never less safe, because a later payment lowers the balance on fewer days. The chance is therefore non decreasing in the payment day, so a binary search finds the earliest day that meets the constraint. A test shows this property (`test_paying_later_is_never_less_safe`). If even the last day fails, the decision is HOLD with no safe date.

2.4 Bill scheduler (`optimiser/scheduler.py`)

OR Tools CP-SAT chooses a release day for each open supplier bill. The chance constraint is written as a scenario constraint over 160 of the simulated futures. For each future `s` and day `t` the balance must stay at or above `B`, unless a binary violation variable `z_s` is set, and the sum of `z_s` may not exceed `floor(alpha * S)`.

The cost of lateness grows faster than linearly, so many short delays beat one long delay. A late payment costs `10 + days + days^2 / 8`, multiplied by a weight that is higher for bills due soon (`1 + 60 // (7 + days_until_due)`) and doubled for protected payees. A bill pushed past 30 days late takes a large fixed cost. The optimiser's plan sets the order in which the Policy Engine considers bills. The Policy Engine still decides the actual date.

2.5 Policy Engine (`engine/policy.py`)

Pure functions. The rule order is kill switch, statutory first, protected payee, duplicate or anomaly, approval limit, mandate, then the chance constraint. A failed hard rule gives DENY, a failed soft rule gives ESCALATE, a failed chance check gives HOLD with the earliest safe date, and otherwise the verdict is ALLOW. Decisions are made in a fixed order and each released payment is committed before the next is judged, so the whole plan meets the constraint and not just each payment. An ESCALATE or DENY is not committed, so a poisoned invoice cannot crowd out real payments. Every decision carries a trace of each check, the numbers behind it, and one sentence in English and Bahasa Malaysia.

2.6 Hash chain (`audit.py`)

Each audit row is `{id, ts, actor, event, payload}` and its hash is `sha256(prev_hash + canonical_json(row))`, using sorted keys and compact separators. `verify_chain` recomputes every hash and reports the first row that does not match. Database triggers refuse updates and deletes by the application on both SQLite and Postgres.

2.7 On-chain anchor (`anchor.py`, `contracts/AuditAnchor.sol`)

The contract has two functions that only emit events, `anchor(seq, headHash)` and `anchorPolicy(version, policyHash)`. It stores nothing, has no payable function and no fallback, so it cannot receive funds. The client sends the hash of the last audit row and a hash of the canonical owner policy and mandate. To verify, the client recomputes the chain from the database, reads the events, keeps only events sent by the expected anchor address, and compares. If a row has been edited, the recomputed hash no longer equals the anchored one and verification fails. Only hashes are written. This is checked by a test that scans every transaction on an in process chain for business words.

2.8 Bank keys (`bank/service.py`)

The Executor mints an HMAC signed key scoped to one destination account, one maximum amount and one payment, valid for five minutes and usable once. The bank rejects anything the key does not cover.

3 Security design

The permission tiers, the threat table with tests, and the known risk list are in SECURITY.md. In short, the Reader has no tools and its output is schema checked, so text inside a document cannot move money. Only the Policy Engine decides and only the Executor mints bank keys. Every state changing route needs an owner token (`auth.py`). The kill switch is reread before every instruction. The Watchdog verifies the chain, checks that every payment traces to an ALLOW decision, watches forecast drift and runs a poisoned invoice self test after every run. It can engage the kill switch and cannot release it.

4 What is real and what is simulated

Part	Status
Forecast, optimiser, policy engine, audit chain, hash anchor client, red team tests	Real code, run for real
Contract bytecode	Compiled from source with solc 0.8.26 and executed on a real in process EVM in the tests
Public test network deployment	Needs a funded throwaway key. The wallet exists in `.env` on the build machine and has no test ETH yet, so no public transaction hash is claimed anywhere
Frankfurter exchange rates	Real calls, saved in `docs/evidence/fx-frankfurter.json`
Bank	Simulated (the Demo bank). A real sandbox needs an account and keys from the owner
Language model	Fixture mode by default. With `LLM_MODE=live` the Reader and Explainer call Gemini (`he lm/llm.py`). Real responses for four documents are recorded in `services/api/tests/fixtures/llm/recorded.json` and replayed in the tests
Business data	Synthetic and labelled so